基于 FPGA 的信号感知加速系统

摘要

无线信号监测需要判断一段 IQ 中有没有信号，以及已知信号属于哪种调制方式。CNN 能够完成这两类分析，但在边缘硬件上，仅有乘加阵列还不足以获得高效推理：窗口展开、数据搬运与逐块调度同样影响执行时间。本作品在 AMD PYNQ-Z2 上构建 RISC-V 与自研加速器协同的信号感知系统，围绕固定 4×8 阵列重新组织供数、片上复用和层内执行，将并行计算能力转化为实际推理效率。

实板对照中，分类在同为 4×8 阵列、逻辑资源几乎相同的条件下，较 im2col＋GEMM 达到 35.06 倍速度；相对所测 hls4ml，两任务 LUT 减少约 70%—79%，CNN 时延同时缩短。该结果体现了自主数据通路在速度与逻辑资源上的组合优势。系统以频谱占用和四类调制识别完成双应用映射，通过网页呈现信号、FPGA 输出与性能对照，并交付四方实板证据和可复用工程。分类分 SNR 表现、占用检出率及量化效果配合硬件结果，展示了从信号分析到专用计算实现的完整设计。

目录

第一部分　作品概述.....3
1.1 功能与特性.....3
1.2 应用领域.....3
1.3 主要技术特点.....4
1.4 主要性能指标.....4
1.5 主要创新点.....5
1.6 设计流程.....5
1.7 选题背景与相关工作.....5
第二部分　系统组成及功能说明.....6
2.1 整体介绍.....6
2.2 硬件系统介绍.....7
2.3 软件系统介绍.....9
2.4 软硬件划分依据与接口设计.....12
第三部分　完成情况及性能参数.....12
3.1 整体介绍.....12
3.2 工程成果.....13
3.3 特性成果.....15
3.4 大模型辅助设计与工程验证.....22
3.5 技能包的提炼与验证.....23
3.6 从零复现操作说明.....24
第四部分　总结.....27
4.1 可扩展之处.....27
4.2 心得体会.....27
第五部分　参考文献.....28

第一部分　作品概述

1.1 功能与特性

系统将频谱占用检测与四类调制识别集成到同一网页平台。用户选择 I、Q 各 128 点的信号片段，提交后可查看波形、散点或功率谱，以及 FPGA 返回的状态、类别与周期。RISC-V 负责启动和控制，自研加速器执行 CNN；两个应用分别映射到同规格 4×8 架构，连接信号观察与硬件结果分析。

1.2 应用领域

在通信实验和边缘无线监测原型中，使用者关心一段 IQ 中是否存在信号，以及已知信号属于哪种调制方式。占用检测给出空闲 / 占用判断，用于观察信号存在状态；调制识别区分 BPSK、QPSK、8PSK 与 16QAM，辅助理解波形的调制特征。系统把两类分析放在统一界面，并呈现对应硬件输出，使信号现象、模型判断和计算代价能够相互对照。

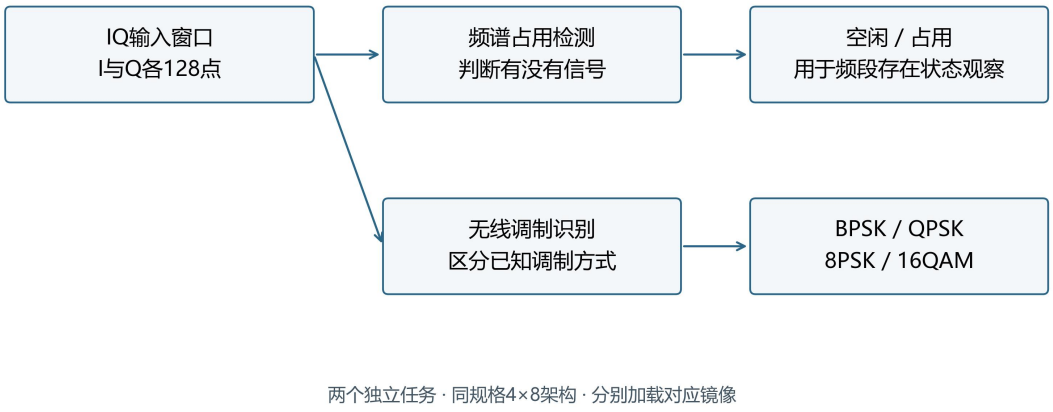


图 1　两类信号感知任务及其分析目标

1.3 主要技术特点

设计围绕持续供数组织固定阵列的执行。按需取窗把卷积窗口直接送入计算通路，避免先展开再搬运；驻留权重和片上激活减少重复访问；层内自主调度把逐块软件控制转为硬件遍历。占用通路以四 bank 读取、A / C 双缓冲和后处理重叠衔接装载、计算与写回，提升固定 32PE 的执行效率。

1.4 主要性能指标

表 1　主要性能指标

指标	结果与评估范围
平台与架构	PYNQ-Z2；RISC-V＋4×8 加速器
精度与时钟	INT8 乘法 / INT32 累加；50 MHz
分类软件效果	全 SNR 68.10%；SNR≥0 dB 97.42%
占用软件效果	INT8 96.68%；FP32 96.94%
占用检出与虚警	INT8 Pd95.65%；Pfa2.30%
CNN 板内周期	分类 44343；占用 35629

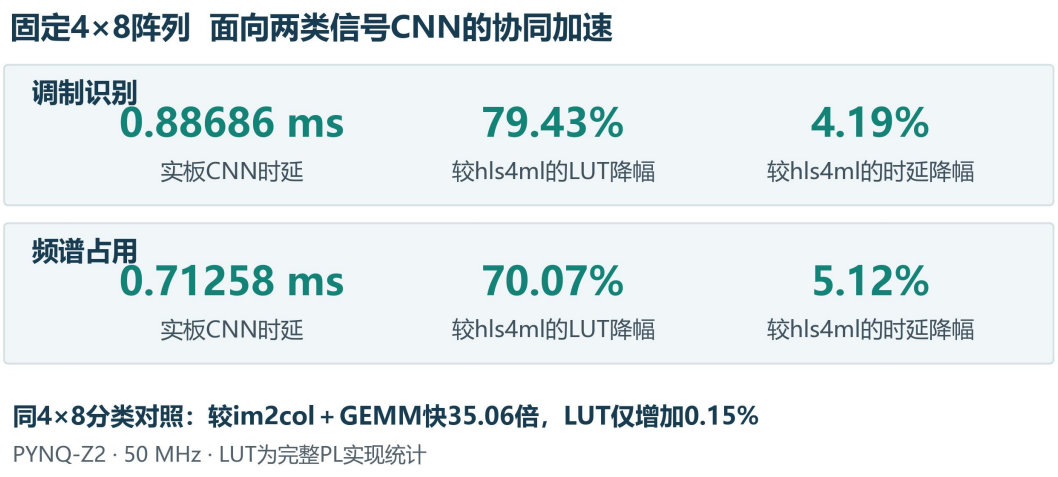


图 2　自研加速器双应用核心成果总览

1.5 主要创新点

（1）一维短序列 CNN 的按需取窗与驻留供数：四 bank 处理窗口起点非对齐，驻留权重提供 8 路数据，减少 CPU 显式展开与打包搬运。

（2）固定 32PE 下的自主层调度与计算重叠：CPU 按层配置，硬件遍历逻辑块；A 预取、乘加与 C 后处理衔接，272 项归约容量覆盖 Conv2 的 270 项。

架构验证：以两类 CNN 映射和四方实板对照，检验同规格架构的适配性与性能。

1.6 设计流程

以信号感知需求确定软硬件分工，冻结模型与整数算术；设计取窗、乘加、片上存储和自主调度，逐层核对数值；完成布局布线及 PYNQ-Z2 实板运行，对比四种实现的速度与资源；最后整合网页演示、工程包和可复用 Skill。

1.7 选题背景与相关工作

本作品关注的问题是：在有限并行规模下，如何让 CNN 计算阵列发挥实际效率。处理器顺序执行密集卷积需要较长时间，但将乘加卸载到硬件后，卷积窗口仍需组织，数据仍需搬运，逻辑块仍需调度。这些环节如果依靠软件逐次处理，便会增加计算之外的开销。因此，设计从乘加单元延伸到供数、存储和控制，让阵列与周边通路共同完成推理。

CNN 的 IQ 识别和整数推理已有研究基础[2,5]。Caffe 将卷积显式展开为矩阵计算[3]；本系统将窗口地址生成和层内遍历纳入硬件，减少 CPU 逐块组织中间数据。所测 hls4ml 采用 Resource、io_stream 与 DATAFLOW 配置[9,11]；自研实现则围绕同一 4×8 阵列安排驻留存储与分块执行。设计贡献在于面向目标一维 CNN 的具体供数与控制组织，并以完整实现结果检验其效果。

四方比较依次回答三个问题：相对 RISC-V 纯软件，专用计算带来多少收益；相对同 4×8 的 im2col＋GEMM，供数与调度设计是否有效；相对 hls4ml，速度与资源如何取舍。两任务以模型评估检验功能，以实板对照检验架构效率。

第二部分　系统组成及功能说明

2.1 整体介绍

系统采用主机交互、RISC-V 控制和专用数据通路分工。主机经 USB/JTAG、VIO-to-AXI 桥把量化 IQ 送入任务邮箱；PicoRV32 读取参数并启动加速器，输出与周期沿同一路径返回。处理器采用开源实现[10]，自研工作集中在 CNN 取窗、存储供数、量化后处理、层内调度及任务接口，使控制灵活性与专用计算各司其职。

图 3 展示控制链路与计算配置。RISC-V 负责描述符、任务启动和状态管理，自研加速器承担 CNN 数据通路。分类配置采用 16 KiB 本地 RAM、窗口字缓存、A 预取与层内调度；占用配置采用 8 KiB 本地 RAM、四 bank 取窗及 A / C 双缓冲。两者均为 4×8、32PE，分别加载对应镜像，分类统计融合由主机完成。

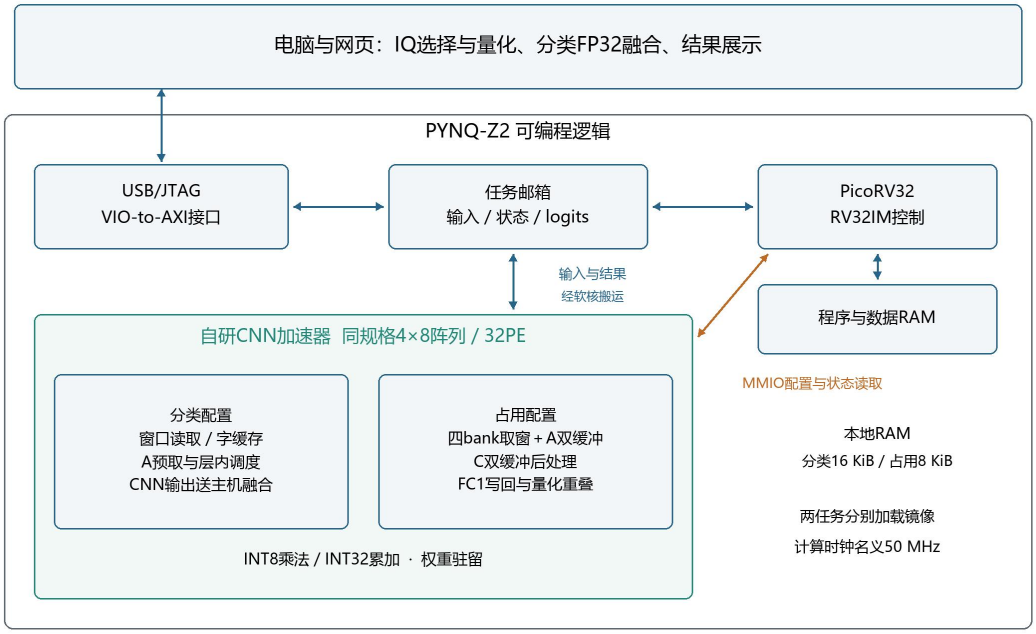


图 3　USB/JTAG 控制链路与两任务计算配置

2.2 硬件系统介绍

2.2.1 硬件整体介绍

演示平台为 PYNQ-Z2，器件 XC7Z020CLG400-1。板载 125 MHz 时钟经 MMCM 产生 50 MHz 计算时钟，主机通过 USB/JTAG 和 Vivado Hardware Manager 完成配置与交互。PicoRV32 按 RV32IM 配置运行，软核侧加速器与任务邮箱地址分别为 0x10000000 和 0x30000000。指令语义参考 RISC-V 规范[1]，处理器实现来源见[10]。

每个 PE 计算有符号 8 位输入与权重的 16 位乘积，符号扩展后反馈到 32 位 ACC 寄存器。start 清空累加器，有效握手时更新，末项有效时锁存 Cij 输出。4 个 A lane 和 8 个权重 lane 向 32 个 PE 广播，构成外积式阵列，实现输入与权重的并行复用。乘加器结合 7 系列 DSP48E1 与逻辑资源实现[7]。

2.2.2 平台连接与时钟

系统采用标准 PYNQ-Z2 开发板，经板载 USB 数据接口下载 PL 配置并交换输入与结果。MMCM 和同步复位逻辑提供计算域时钟与启动条件。第三部分给出板卡实物、供电及 USB 连接照片。

2.2.3 电路各模块介绍

输入窗口模块按位置、通道及卷积参数生成地址，将 Q 激活分布到四个独立字节 bank 并重排，一次同步读取取得非对齐窗口数据。A 双缓冲在当前 tile 计算时装入下一 tile。驻留权重采用 32 KiB 与 8 KiB 分段双端口 BRAM，保存按硬件布局打包的 38880 字节权重镜像，直接向阵列提供 8 路权重，减少 B tile 复制。

GEMM 控制器管理归约、部分和与写回。C 双缓冲使卷积结果的偏置、舍入移位、饱和及按层 ReLU 与后续计算重叠，生成 INT8 激活并写回 Q；Conv2 保留有符号输出。FC1 采用写回与量化流水重叠，最终 FC2 沿末层通路保留 INT32 logits，再由软核经邮箱发布。

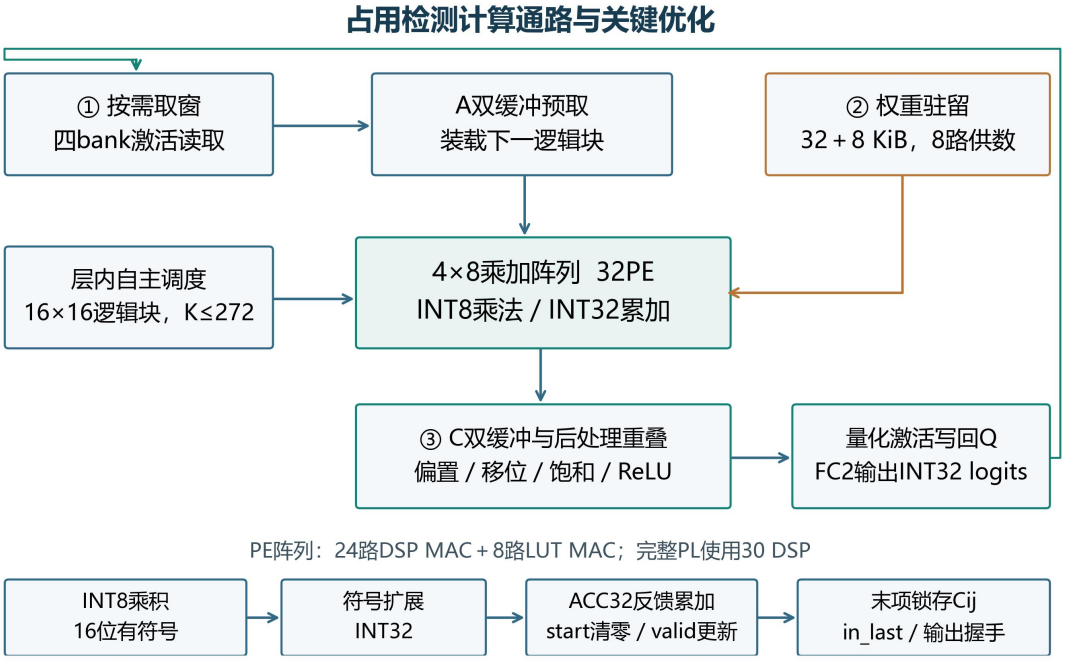


图 4　占用检测计算通路与三个关键优化位置

图 4 中的组合对应目标 CNN 的具体形状：四 bank 取窗处理窗口起点非对齐，驻留权重提供 8 路数据，A 预取与 C 后处理缓冲衔接计算。Conv2 需归约 45×6＝270 项，容量 272 可覆盖该层，减少分段启动；FC1 仍按逻辑块执行，并重叠写回与量化。表 2 和四方实测共同检验供数、存储与调度的联合效果。

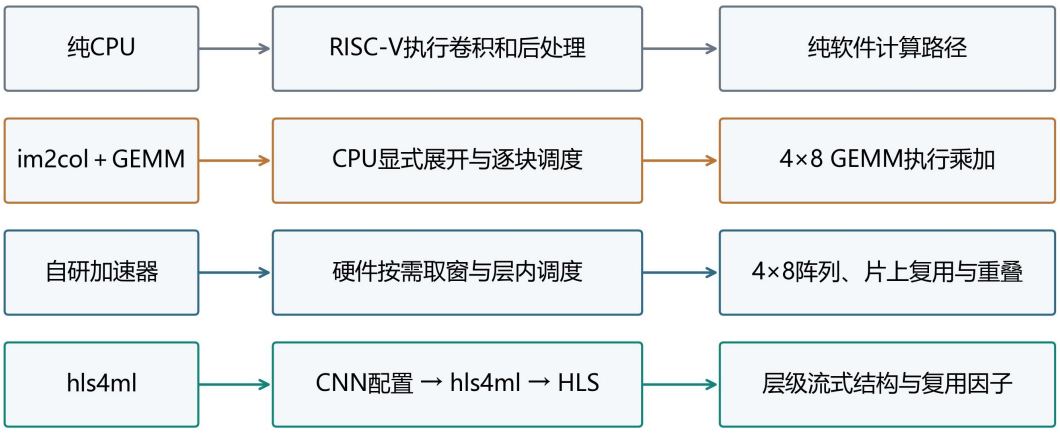


图 5　同一 CNN 的四种实现路径与职责差异

表 2　数据通路组织与显式展开对照

结构措施	im2col 移植中的开销	自研架构设计
按需取窗与驻留	CPU 显式展开、写入和读取 tile	窗口硬件供数，复用片上激活和权重
层内自主调度	CPU 逐块配置与轮询	按层配置，硬件遍历逻辑块
A / C 缓冲重叠	装载、计算与处理串行等待	在依赖允许时重叠；增加控制与缓冲
占用归约容量 272	Conv2 的 270 项拆分归约	一次覆盖，减少重复启动
分类实现级结果	1554656 周期，7473 LUT	44343 周期，7484 LUT；联合设计结果

表 3　关键硬件接口与控制信号

接口或信号	关键含义
AXI AW / W / B 和 AR / R	地址、写数据独立接收；响应在反压期间保持稳定
输入窗口与权重端口	位置 / 通道 / 核参数，片上激活及权重地址
start / valid / ready / in_last	清零、有效数据握手、末项锁存
busy / done / error	执行状态、完成与非法访问结果
请求 ID 与模型 / 固件身份	核对结果对应任务及加载版本，防止误用旧数据

2.3 软件系统介绍

2.3.1 软件整体介绍

软件分为模型训练评估、整数参数导出、RISC-V 控制固件、主机驱动及网页展示层。CNN 从复数 IQ 中学习信号特征[2]，两个应用分别冻结模型和评估流程。主机对输入量化，接收真实 FPGA 返回的整数输出并与参考比较，再绘制结果；分类最终融合沿用主机 FP32 统计支路。

主机通过 USB/JTAG、VIO 桥与 AXI 任务邮箱交互，上传 256 字节量化 IQ 并读取 FPGA 整数输出、状态与板内计数。模型身份与请求编号用于配对输入和结果；任务切换加载对应镜像。通用 PYNQ Overlay 预检技能作为技能包的独立复用项交付[4]。

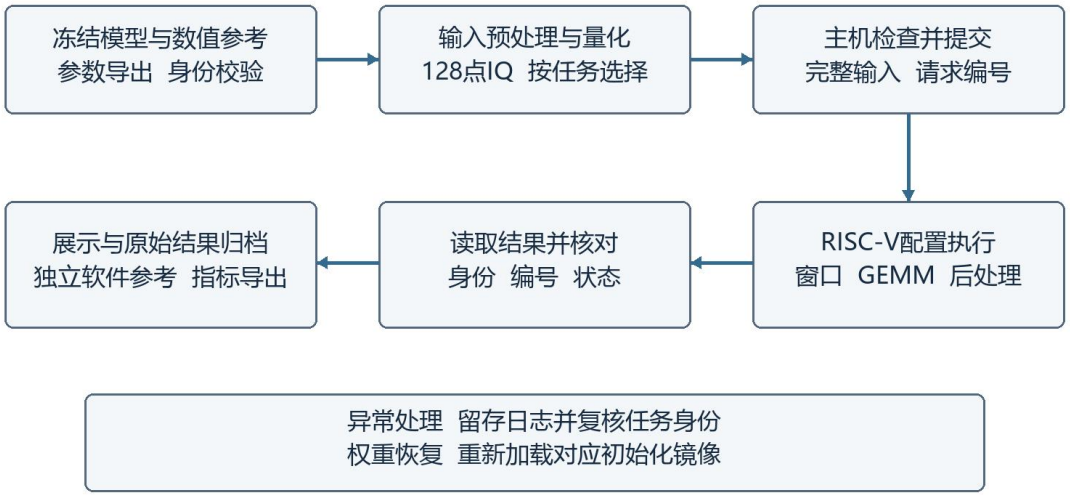


图 6　从输入校验到硬件结果返回的软件流程

2.3.2 软件各模块介绍

调制识别沿用 RadioML2016.10A 四类共 80000 条样本，训练、验证和历史测试划分分别为 55956、12062 和 11982 条，输入为 2×128 I/Q。INT8 CNN 与主机 FP32 统计残差支路共同输出最终类别。占用检测使用有完整生成记录的可控信号与复高斯纯噪声数据，在 66000 帧均衡仿真留出集上评估；全局裁剪与量化尺度固定，保留帧间能量差异。

分类 CNN 由三层一维卷积构成，通道为 2→32→64→64，卷积核为 7、5、3。前两层经 ReLU 及 2 倍最大池化，第三层后进行全局平均池化和 64→4 全连接；BatchNorm 参数在推理前折叠。主机统计支路提取 IQ 循环相关、非共轭相关及高阶频谱特征，经标准化和小型全连接残差头后，与缩放后的整数 CNN logits 相加输出类别。

表 4　冻结占用网络及量化处理

占用网络层	权重或输入形状	输出与处理
输入	2×128	通道先 I 后 Q
Conv1	45×2×8	45×121；shift 8；ReLU
Conv2	9×45×6	9×116；shift 8；保留负值
FC1	32×1044	32；shift 12；ReLU
FC2	2×32	2 个 INT32 logits

参数与固件模块：导出 INT8 权重、INT32 偏置及移位描述符，生成初始化文件并核对源身份。固件配置卷积层描述符，由硬件完成逻辑分块遍历、窗口装载和链式后处理；全连接按对应层配置执行。模型及量化参数沿用冻结版本，更新后的占用固件为 2276 字节。类别规则为 logit1 大于 logit0 判占用，否则判空闲。

邮箱与驱动模块：占用 ABI2 输入为 64 个 32 位字，按字节有效位累计，收齐 256 字节后启动；主机写入请求编号，执行期间锁定输入与启动接口。完成后读取两个 logits、周期和阶段计数，并核对请求编号与复位代数。异常恢复采用日志留存、复位或镜像重载与身份复核流程；权重异常通过重新加载初始化镜像恢复。

验证模块用 NumPy 整数参考、C 实现和 RTL 逐层核对激活、摘要及 logits，并检查输入完整性、总线反压及复位。四方计算实现包括 RISC-V 纯软件、Caffe im2col 思路移植后连接 4×8 GEMM、hls4ml 1.3.0 生成硬件，以及自研加速器。它们执行同一任务的冻结 CNN，为不同数据与控制组织提供对照。大模型辅助源码分析、候选设计、测试脚本和工具排错，开发者结合真实输出、周期和实现报告作设计取舍。第 3.4 节展示典型协作与决策，第 3.5 节介绍由项目经验提炼的可复用 Skill。

2.4 软硬件划分依据与接口设计

软硬件划分以减少密集计算中的软件介入为原则。FPGA 执行乘加、窗口供数、层内分块遍历及量化后处理；PicoRV32 按层配置描述符并管理任务状态；主机完成输入选择、量化、分类 FP32 融合和可视化。两任务使用同规格 4×8 架构的独立实现，分别保存权重、功能配置与部署镜像。

主机接口使用 32 位 AXI-Lite，地址和写数据独立握手，读写响应在反压期间保持。邮箱偏移 0x00/0x04 为 magic/ABI，0x08 为状态，0x10 为控制，0x14/0x18 为请求/结果 ID，0x1C 为错误码。0x100～0x1FC 容纳 64 个输入字；0x40 起为 INT32 logits，0x50 为周期，0x60 起为模型 SHA。分类 ABI1 返回四个 logits，占用 ABI2 返回两个，并扩展输入长度、复位代数和阶段计数。

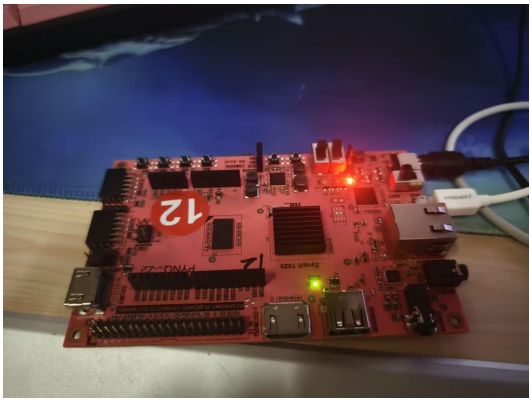
调用顺序为身份核对 → 等待空闲 → 上传 256 字节 → 写请求 ID 并启动 → 等待完成 → 校验结果 ID 和错误状态 → 读取输出及计数 → 确认完成。忙时禁止覆盖输入，输入未齐不启动；复位代数用于识别请求中途的复位。控制恢复与模型存储恢复分开处理，镜像重载后重新核验身份和黄金输出。

第三部分　完成情况及性能参数

3.1 整体介绍

系统已完成两类模型评估、整数映射、RISC-V 控制、RTL 验证、布局布线及 USB/JTAG 实板运行。图 7 为板卡实物与连接。两个应用使用同规格架构的独立镜像，主机负责输入、参数组织与结果展示，板上处理器和专用阵列负责 CNN 控制与计算。

项目以模型软件评估、逐层数值检查、协议验证、完整 PL 实现和实板计数构成验证链路。CPU 补充对照每任务完成 4 次板上调用，分类使用 1 个固定输入、占用使用 2 个，计数与对应仿真一致。全量软件评估用于衡量识别效果，硬件样例与协议测试用于确认计算实现和交互正确性。



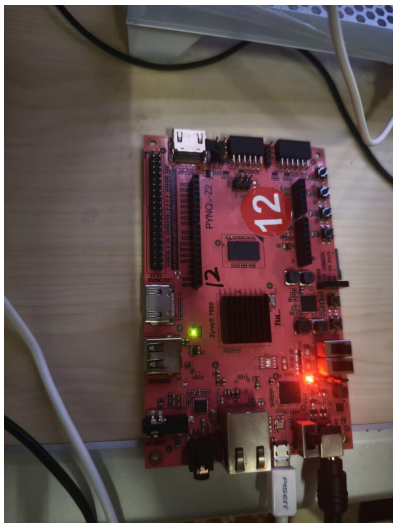


图 7　PYNQ-Z2 板卡实物与供电、USB 数据连接

3.2 工程成果

3.2.1 平台部署成果

主工程包含两任务的动态输入镜像，以及自研、hls4ml、im2col 三种实现的六个基准镜像。CPU 补充工程另提供两个不含 CNN 阵列的完整 PL 镜像，并归档其源码、程序、bit / LTX 及运行结果；生产加速器与模型保持冻结。

3.2.2 电路成果

电路实现包含处理器、存储、CNN 数据通路、时钟与测试接口。各基准工程均完成布局布线，资源表统一统计完整 PL，与相应固定输入的板内计数配对。CPU 纯软件工程的 CNN 实例数为 0，完整 PL 使用 4 个 DSP；其他实现按各自的数据通路映射完成计算。

表 8 集中给出两任务四种实现的资源与时序结果，图 14 和图 15 展示各项资源的绝对数量，便于同时比较速度、逻辑规模与存储开销。

3.2.3 软件成果

网页提供信号分析、性能对照和硬件架构展示。图 8、图 9 为真实 FPGA 交互：分类 / 占用动态接口为 44381 / 35629 周期；约 37—40 s 的 USB / 配置耗时单独记录。

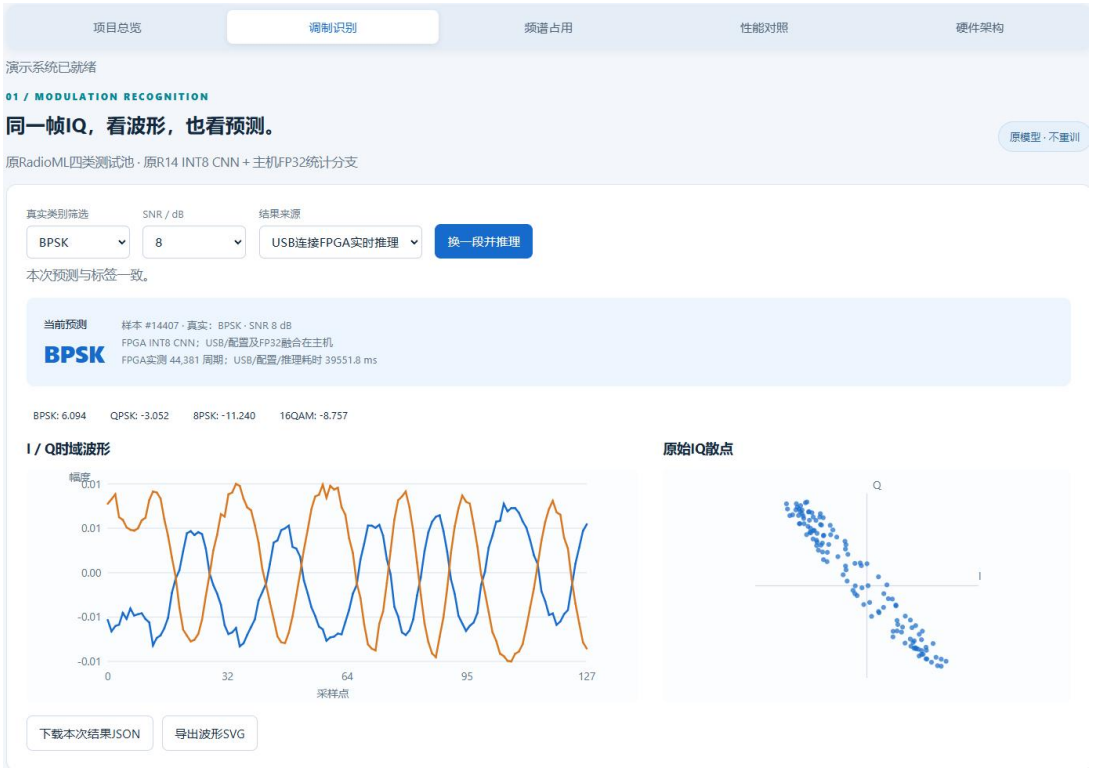


图 8　调制识别在线交互结果与同帧 IQ 展示



图 9　频谱占用在线交互结果与同帧波形、功率谱

3.3 特性成果

四方实板对照表明，自研加速器的优势不只是将 CNN 从处理器移到硬件，而是在固定阵列下提高执行效率，并以较少逻辑资源获得比所测 hls4ml 略短的推理时间。图 10 比较两任务的板内 CNN 时延，图 11 将时延与 LUT 放在一起，展示速度与资源的组合效果。

RISC-V 纯软件体现专用计算收益，im2col＋GEMM 检验同规模阵列下的供数与调度价值，hls4ml 评估自主数据通路与自动映射的实现差异。四方执行各任务的冻结 CNN，按 50 MHz 板内计数换算时延；详细配置与计时范围集中列于 3.3.5 节。

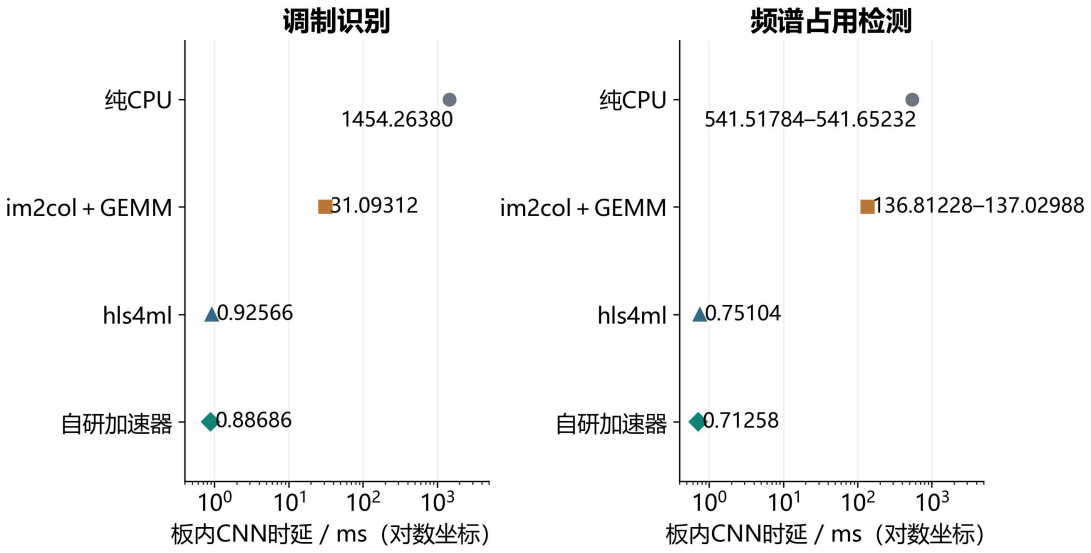


图 10　两任务四种实现的实板 CNN 时延总览

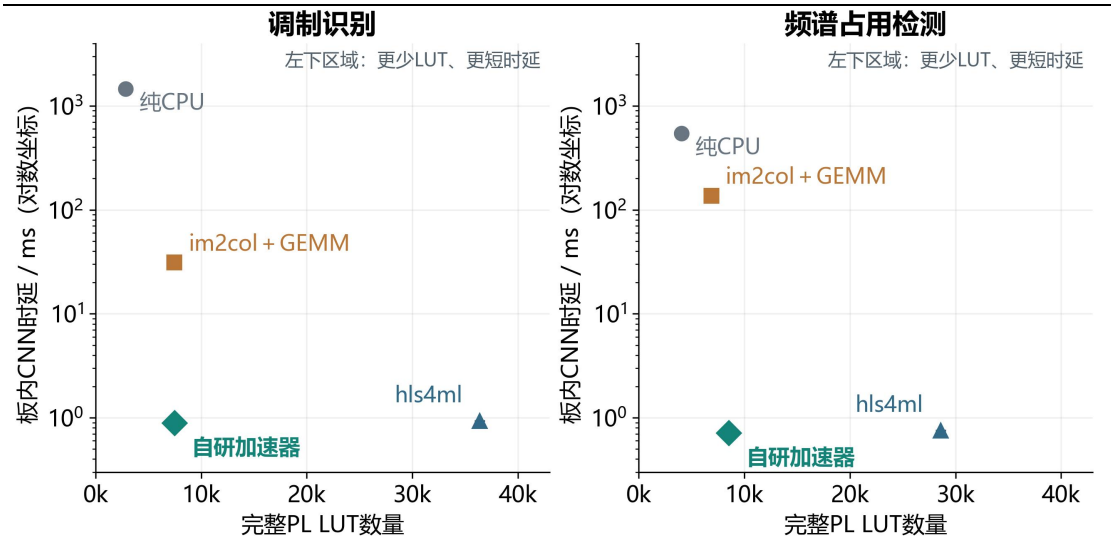


图 11　四种实现的时延与 LUT 联合对照

图 10 显示从纯软件到硬件计算的时延变化；占用结果的横向范围对应两个输入的周期差异。图 11 采用一致的坐标范围及对数时延轴，自研实现相对所测 hls4ml 位于更少 LUT、更短时延的一侧。同 4×8 的 im2col＋GEMM 与自研实现则展示：阵列规模接近，并不意味着实际执行效率相同。

3.3.1 模型准确率与数值一致性

表 5　冻结模型的软件评估结果

任务及评估范围	结果
四类分类混合模型　11982 段	8160 段正确；准确率 68.10%
分类　SNR≥0 dB 子集	准确率 97.42%
占用 INT8　独立仿真留出集 66000 帧	准确率 96.6758%；Pd95.6515%；Pfa2.30%
占用 FP32　同一留出集	准确率 96.9394%; Pd95.9303%; Pfa2.0515%

软件评估衡量感知效果，整数核对确认硬件实现，两者承担不同的验证任务。分类采用 INT8 CNN 与主机 FP32 统计支路融合，图 12 展示 11982 条历史测试样本的分 SNR 表现与混淆矩阵，表 5 保留全 SNR 及高 SNR 结果。图 13 同时呈现融合贡献与占用量化影响：占用 INT8 准确率较同留出集 FP32 下降约 0.26 个百分点。Pd 按占用样本 SNR 统计。

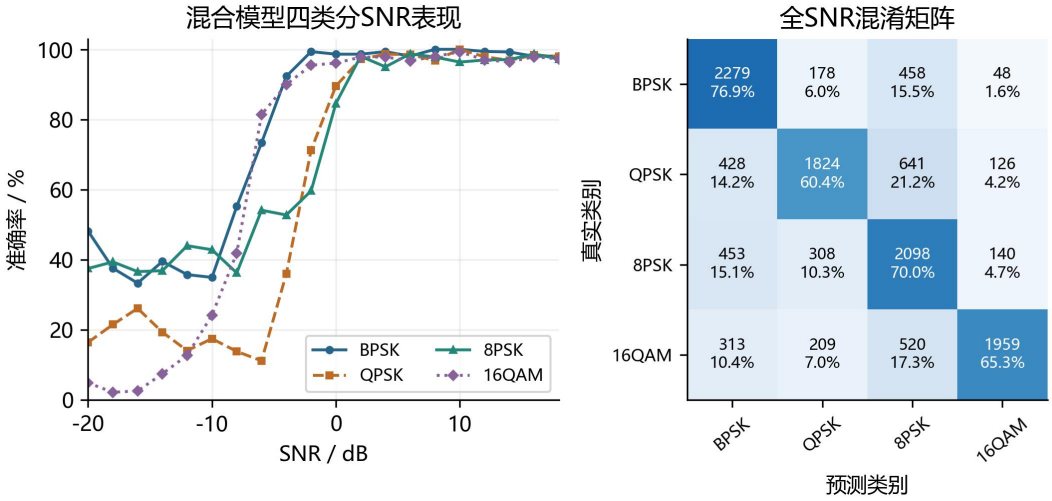


图 12　分类分 SNR 准确率与混淆矩阵

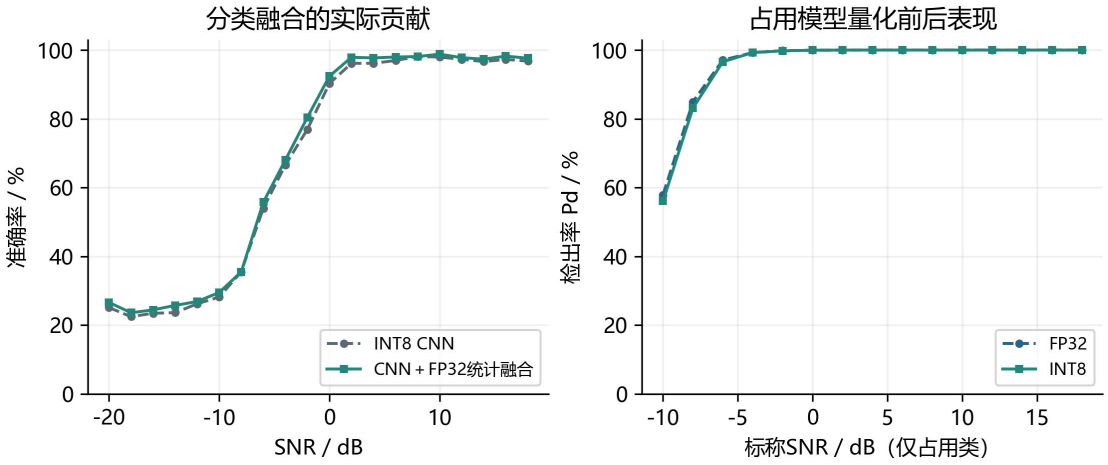


图 13　分类融合贡献与占用 FP32 / INT8 检出率

新占用实现的 30 个唯一冻结输入加一次复位重放，共 31 次调用、93 个逐层 FNV 摘要和 62 个最终 logits 全部匹配，每次 35629 周期。gather 单元通过 18 场景、41932 字节检查；14 个非法层描述符均被拒绝且无计算或写回副作用。USB 顶层及恢复重放通过，实板冻结输入 00 / 29 各运行两次，四次周期与最终输出均一致。上述数值核验沿用冻结模型；表 5 为独立的软件全量评估。

3.3.2 调制识别 CNN 四种实现速度对照

表 6　调制识别 CNN 四方实板周期对照

分类实现	板内周期	50 MHz 换算 ms	DSP
CPU 纯软件	72,713,190	1454.2638	4
im2col＋GEMM	1,554,656	31.09312	36
hls4ml	46,283	0.92566	68
自研加速器	44,343	0.88686	36

同为 4×8 阵列、16 KiB 处理器 RAM，自研与 im2col＋GEMM 的 LUT 分别为 7484 和 7473，分类 CNN 时延却由 31.09312 ms 降至 0.88686 ms，达到 35.06 倍速度。这个对照说明，仅比较乘加阵列规模不足以判断推理效率，供数与控制组织同样重要。

自研实现以硬件取窗、片上复用和层内自主调度衔接计算，实测反映整套设计的联合收益。相对目标 RISC-V 纯软件达到 1639.79 倍速度；相对所测 hls4ml 时延降低 4.19%。

3.3.3 频谱占用 CNN 四种实现速度对照

表 7　频谱占用 CNN 四方实板周期对照

占用实现	板内周期范围	50 MHz 换算 ms
CPU 纯软件	27,075,892～27,082,616	541.51784～541.65232
im2col＋GEMM	6,840,614～6,851,494	136.81228～137.02988
hls4ml	37,552	0.75104
自研加速器	35,629	0.71258

占用检测进一步检验该设计在另一种 CNN 结构中的实现效果。自研通路衔接四 bank 取窗、驻留权重、A / C 缓冲及量化后处理，在两组板测输入中均为 35629 周期。相对所测 hls4ml，LUT 减少 70.07%，CNN 时延降低 5.12%，体现了较小逻辑规模与较短推理时间的组合优势。四方完整计数见表 7。

3.3.4 资源优化与实现时序

表 8　两任务四种实现的完整 PL 资源与时序

完整 PL 实现	LUT	FF	BRAM36	DSP	setup ns	hold ns
分类 CPU	2827	3541	32	4	10.152	0.049
分类 im2col	7473	7349	18.5	36	2.951	0.024
分类 hls4ml	36386	42260	17	68	2.288	0.007
分类 自研	7484	7350	18.5	36	3.020	0.056
占用 CPU	4054	4653	32	4	8.055	0.037
占用 im2col	6881	6615	53.5	36	6.392	0.016
占用 hls4ml	28586	46590	20	46	3.221	0.010
占用 自研	8556	8019	20	30	4.824	0.030

将图 11 的时延与表 8 的资源对应起来，可以看到自研实现并非依靠大幅扩大逻辑规模换取速度。分类与占用分别使用所测 hls4ml 约 20.57%和 29.93%的 LUT，板内推理时间也更短；FF 与 DSP 同样减少。BRAM、时序余量及各方完整资源同时列示，比较覆盖处理器、存储与计算通路，而非仅统计乘加阵列。

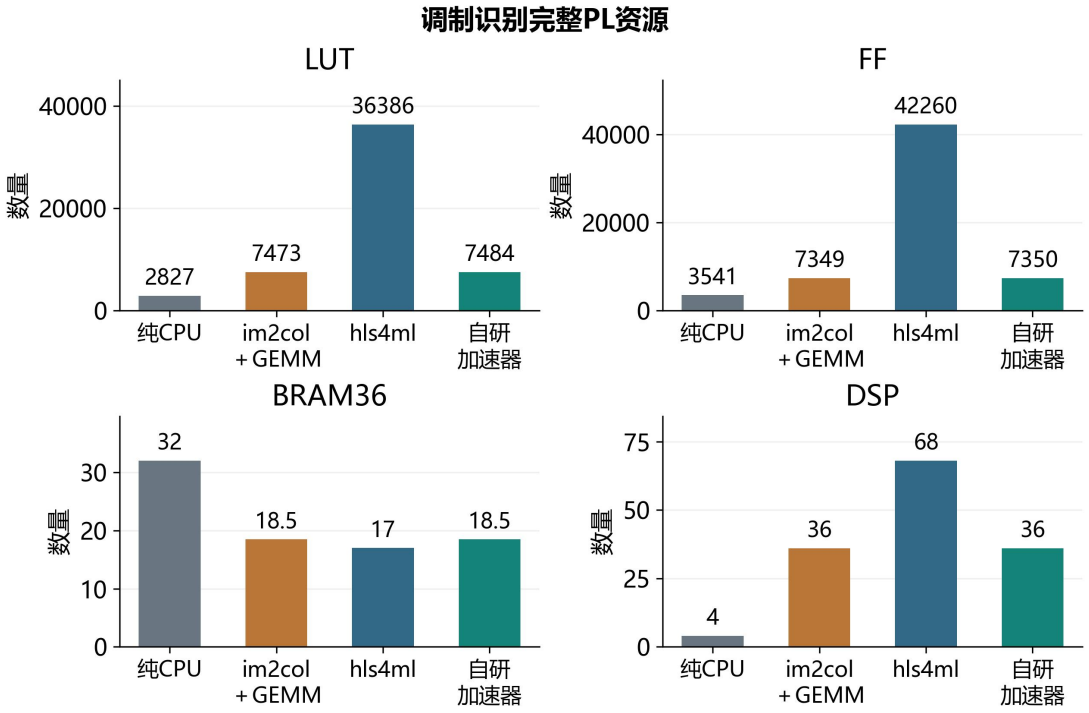


图 14　调制识别四种实现的完整 PL 资源

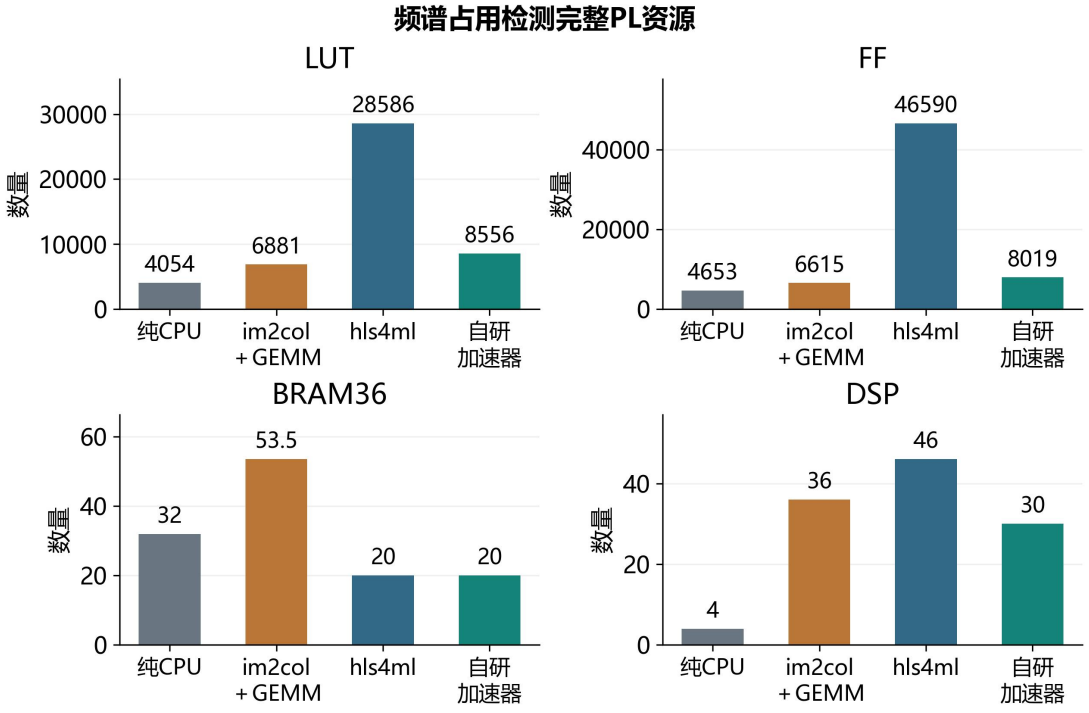


图 15　频谱占用四种实现的完整 PL 资源

3.3.5 测试配置与验证记录

性能对照在 PYNQ-Z2、XC7Z020CLG400-1、50 MHz 条件下进行，采用 Vivado／XSim 2020.2、Vitis HLS 2020.2 及 hls4ml 1.3.0。RISC-V 配置为 RV32IM、fast MUL、无硬件 DIV，固件使用 xPack GCC15.2.0-1 的-Os 配置。

图 16 将板内计数与主机交互分开。分类 CPU 汇总输入复制、三卷积及聚合分类头五段 rdcycle 区间，摘要检查另计；分类硬件采用对应基准的 CNN 区间。占用四方统计量化邮箱输入复制至最终 logits。输入与权重预载、USB/JTAG 及主机处理均在上述区间之外。

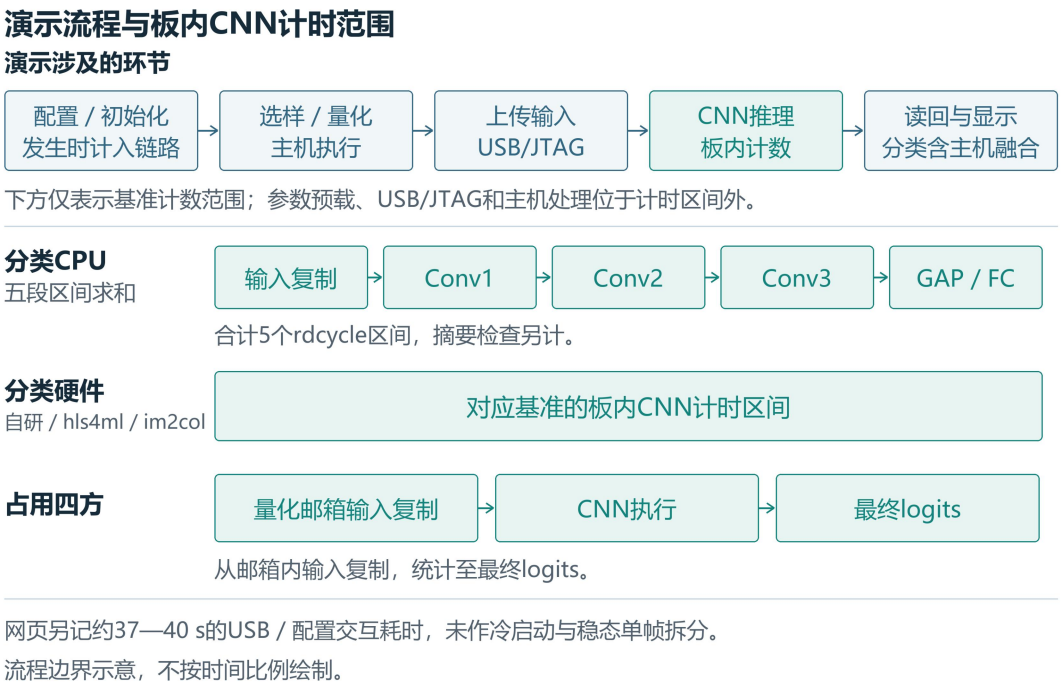


图 16　演示流程与板内 CNN 计时边界

网页约 37—40 s 为另记的 USB／配置交互耗时，不作为稳态单帧时延；图 16 只表示流程边界，不按时间比例绘制。

CPU 纯软件均配置 128 KiB 本地 RAM；分类另三项为 16 KiB，占用 im2col 为 128 KiB、自研与 hls4ml 为 8 KiB。

hls4ml 采用 Resource 策略、io_stream 与 DATAFLOW，分类卷积及 FC 复用因子为 28／640／768／16，占用为 48／270／2088／32[11]。四方为实际配置的实现级比较，各项资源和时延同时列示。

板测记录中，分类 CPU 为 4 次、自研与 hls4ml 各 2 次、im2col 为 1 次；占用自研、CPU 与 im2col 各 4 次。占用 hls4ml 按两输入归档 37552 / 37552 周期并标注总调用数 4。CPU 采用目标 PicoRV32 及手工四输出分块 C；输出、程序及镜像身份随证据保存。

CPU-only 分类的四次实板计数均 72713190，独立仿真两次同值，并与黄金值核对 8 个层摘要和 8 个 logits；占用四次板测与两次仿真分别匹配 27075892 / 27082616。综合确认两 CPU 工程均无 CNN 实例、DSP 为 4，完整实现 setup 和 hold 为正、路由错误为 0。CPU 补充包保存实际命令、输出、源码、程序与镜像身份，原有三种实现的已确认板测记录直接复用。

四方性能来自正式干净工程及 CPU 补充资料，软件识别效果来自冻结模型的评估结果。模型、输入、固件、镜像和实现报告均以身份清单关联，组成可核对的证据链，支撑架构比较、结果复查与工程交接。

3.4 大模型辅助设计与工程验证

本项目将大模型辅助贯穿源码分析、候选实现、测试脚本编写与工具排错，形成“约束定义—方案生成—工程验证—结果反馈”的迭代开发流程。开发围绕既定 CNN 模型、整数计算规则和 4×8 并行规模展开，通过仿真、布局布线及实板结果评价候选方案，使设计迭代与工程验证相互衔接。

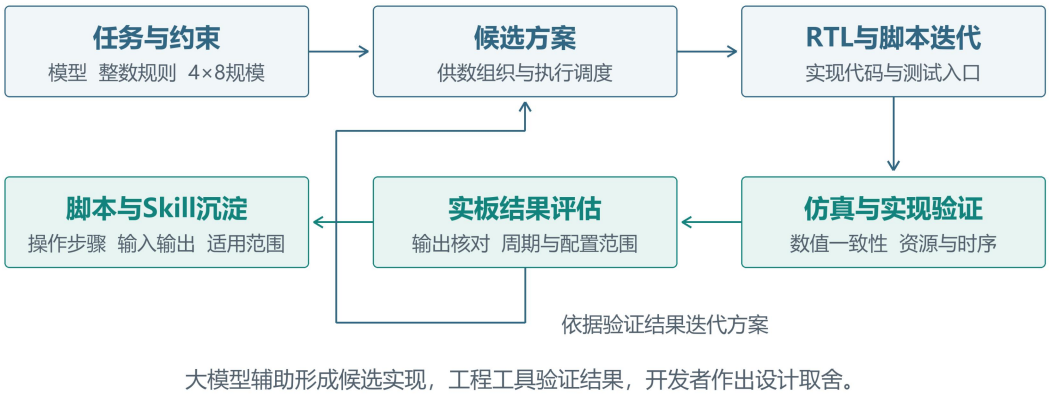


图 17　大模型辅助设计的研发流程

（1）面向固定计算规模的架构优化。针对占用 CNN 中供数、计算与后处理的衔接问题，大模型辅助分析数据通路和控制流程，参与候选 RTL 实现与测试脚本准备。最终设计在 32PE 规模下采用四 bank 按需取窗、权重驻留、A 预取与 C 后处理缓冲，并通过层内自主调度减少 CPU 逐块组织任务的开销。方案取舍结合数值一致性、实现资源和实板周期进行判断，形成面向目标一维 CNN 的供数与执行组织。

（2）面向多种实现的验证与结果整理。围绕纯 CPU、im2col＋GEMM、hls4ml 和自研加速器，利用大模型辅助整理测试入口、核对配置并分析工具输出。通过逐层整数结果比对、模型与镜像身份检查，以及明确的计时范围，建立从计算正确性到硬件性能的验证链，支撑两项任务的四方对照。

（3）将研发经验沉淀为可复用技能。对整数推理核验、部署预检、资源时序分析和实验取证等重复工作，整理操作步骤、输入输出、适用范围与常见问题，形成配套脚本和 Skill。路径问题采用失败复现、修正自测与回滚复验确认，将一次性排错经验转化为后续工程可沿用的方法。

上述内容依据研发记录归纳整理，相关交互选段、脚本和验证结果随工程资料保存。

3.5 技能包的提炼与验证

技能包以 skill/README.md 为入口，包含两个可独立运行的脚本技能和三个流程技能。edge-int8-validation 检查整数边界及实现摘要，

pynq-overlay-preflight 检查 bit/HWH、IP 映射与 MMIO 边界；其余三个流程技能分别组织实验取证、整数等价和资源时序分析。接收者可按任务选择使用，不必运行本项目的 CNN。

技能包将本项目的整数边界、镜像配对、资源时序分析及实验记录经验转化为可直接复用的步骤、模板和脚本。每项说明适用场景、操作方式、验证效果与失效条件，支持其他 CNN 或 MMIO 工程沿用检查方法，减少重复定位和证据整理工作。

两项脚本技能分别通过 14 项和 13 项主机自测，覆盖整数值 / dtype / shape、非有限数、缺字段、错误 SHA、错误 IP 及越界访问；三个流程技能提供试验记录、数值等价及资源时序分析模板。通用 PYNQ 技能的对象测试与本作品 USB/JTAG 板测属于不同证据范围。

技能使用边界集中写入 README：摘要检查用于快速定位，完整 DRC 与约束覆盖由实现工具确认；主机对象测试用于接口预检，板级功能由实际运行核验。两项脚本共 27 项自测，相关结果及 Windows 路径修正记录位于 skill/evidence 和 report/skill_fix。

3.6 从零复现操作说明

3.6.1 准备工程与环境

主工程以公开仓库 https://github.com/wudidamaozi/signal_sensing_accelerator 的冻结干净包为准，CPU 实板对照由配套 cpu_baseline_results_20261101.zip 补充。复现环境为 Windows PowerShell、Python3.12、NumPy2.4.6 及 Vivado / Vitis HLS2020.2，工具目录按本机位置设置。以下命令在英文路径的工程根目录执行。

```
Set-Location 'D:\signal_sensing_accelerator'
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
Set-Alias python "$PWD/.venv/Scripts/python.exe"
$env:EDGEAI_VIVADO_BIN='D:/Xilinx/Vivado/2020.2/bin'
```

安装 USB Cable Drivers，并按本机位置设置 Vivado 路径。离线软件检查使用包内模型和黄金样本；完整训练数据另按来源许可获取。复跑构建或仿真时使用全新输出目录，保留已有归档结果。

3.6.2 校验冻结资料与技能包

```
python sim/smoke.py --output build/output/offline_check
python skill/edge-int8-validation/scripts/self_test.py
python skill/pynq-overlay-preflight/scripts/self_test.py
```

成功标记为 CLEAN_PACKAGE_SMOKE_PASS，覆盖冻结清单、30 个占用黄金输入、两任务参考与 27 项技能测试。已执行的独立解压迁移结果保存在 report/package_validation.json，便于对照检查。

3.6.3 重跑 RTL 并构建动态镜像

两任务动态接口各提供两个输入的仿真。先运行 RTL 检查，再按需要综合、实现。入口从 build/variants.json 读取对应源码与测试台，使用工程包内的初始化文件。

```
python sim/run_xsim.py --variant occupancy_live `
  --vivado-bin $env:EDGEAI_VIVADO_BIN --output build/output/occupancy_sim
python sim/run_xsim.py --variant classification_live `
  --vivado-bin $env:EDGEAI_VIVADO_BIN --output build/output/classification_sim
```

迁移验证记录给出占用 35629 / 35629 周期、分类 44381 / 44381 周期及 PASS_RELOCATED_XSIM 标记。以下命令构建 bit，输出至 build/output 对应 variant 目录。

```
python build/run_build.py --variant occupancy_live `
  --vivado-bin $env:EDGEAI_VIVADO_BIN
python build/run_build.py --variant classification_live `
  --vivado-bin $env:EDGEAI_VIVADO_BIN
```

3.6.4 连接实板与网页展示

为 PYNQ-Z2 供电并连接 USB 数据线，Vivado Hardware Manager 应识别 XC7Z020。保持一块目标板连接，通过--program 下载易失 PL 配置，Flash 保持不变。默认板上入口使用 board/live 配对的归档镜像。

```
python board/run_board.py --task occupancy --program `
  --output build/output/occupancy_board.json
python board/run_board.py --task classification --program `
  --output build/output/classification_board.json
python build/run_web.py --port 8771
```

入口核对模型 / 镜像 SHA 与返回 logits，通过后输出 REAL_FPGA_PASS。动态接口参考周期为占用 35629、分类 44381；网页地址为 http://127.0.0.1:8771/。网页性能竞速按归档板测比例回放，在线推理返回实际 FPGA 结果；主机与 USB 链路耗时单独记录。

3.6.5 对照构建及交付验证范围

hls4ml 两任务的 C++、冻结 RTL 及权重均在包内。六个基准 variant 由 classification / occupancy 与 native / hls4ml / im2col 组合命名，可交给统一构建与仿真入口。以下命令重新生成 HLS RTL。

```
python build/run_hls.py --task occupancy `
  --hls-bin 'D:/Xilinx/Vitis_HLS/2020.2/bin'
python build/run_hls.py --task classification `
  --hls-bin 'D:/Xilinx/Vitis_HLS/2020.2/bin'
```

占用固件可用 build/compile_occupancy.py 及 xPack RISC-V GCC15.2.0 的 riscv-none-elf-前缀复编译，程序要求生成二进制 SHA 与冻结身份一致。工具安装目录和编译器不随源码包分发，详细参数见 build/README.md。

CPU 补充资料采用独立入口，保留源码、程序、bit / LTX、实板日志和匹配仿真。使用时按本机目录调整脚本中的研究路径并核验 manifest。分类参数和占用 C 二进制身份已核对，主工程迁移验证见 report/package_validation.json。

归档板测使用 board/live 配对镜像，新构建输出位于 build/output。验证重建结果时，在独立副本中更新同次构建的 bit、LTX 和身份清单，再核对黄金输出与周期。

第四部分　总结

4.1 可扩展之处

系统可沿统一描述符与控制接口扩展其他一维信号 CNN，并在既有基础上发展连续输入、快速任务切换和更丰富的可视化分析。后续结合实采数据、端到端响应与功耗测量，可进一步面向实际边缘监测节点完善部署。

4.2 心得体会

本项目最重要的收获，是认识到乘加阵列的规模并不直接等于实际推理效率。相同 4×8 阵列在不同供数与控制方式下产生明显时延差异，促使我们把设计重点从运算单元本身，转向窗口组织、片上复用与执行调度的协同。

研发中，我们保持目标并行规模，以取窗、驻留和缓冲设计改善计算衔接，再用完整实现资源与实板周期判断方案。逐层整数核对保证计算对应冻结模型，四方对照则帮助区分专用计算、自主数据流与自动硬件映射的效果，避免只凭单项数字作设计判断。

大模型辅助候选分析、脚本编写与工具排错，开发者通过真实结果作取舍，并把可重复使用的方法整理为 Skill。工程包、配置和验证记录让这些结论能够被检查，也让后续模型映射有据可循。

最终，系统把两类信号分析、RISC-V 控制、自研计算通路与网页展示连接起来。作品的核心价值，是在有限硬件规模下把数据供给、计算与控制组织成高效的执行过程，并以双应用实板结果证明这一设计的实现价值。

第五部分　参考文献

[1] WATERMAN A, ASANOVIC K, eds. The RISC-V Instruction Set Manual Volume I Unprivileged ISA, Version 20191213[EB/OL]. RISC-V Foundation, 2019[2026-10-31]. https://docs.riscv.org/reference/isa/v20191213/_attachments/riscv-unprivileged.pdf.

[2] O'SHEA T J, CORGAN J, CLANCY T C. Convolutional Radio Modulation Recognition Networks[EB/OL]. 2016[2026-10-31]. https://arxiv.org/abs/1602.04105.

[3] JIA Y, SHELHAMER E, DONAHUE J, et al. Caffe: Convolutional Architecture for Fast Feature Embedding[EB/OL]. 2014[2026-10-31]. https://arxiv.org/abs/1408.5093.

[4] PYNQ. Overlay, PYNQ Documentation v3.0.0[EB/OL]. [2026-10-31]. https://pynq.readthedocs.io/en/v3.0.0/pynq_libraries/overlay.html.

[5] JACOB B, KLIGYS S, CHEN B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[C]//CVPR. 2018: 2704-2713. https://arxiv.org/abs/1712.05877.

[6] XILINX. Vivado Design Suite User Guide: Synthesis, UG901 v2020.2[EB/OL]. 2021-01-28[2026-10-31]. https://docs.amd.com/v/u/2020.2-English/ug901-vivado-synthesis.

[7] XILINX. 7 Series DSP48E1 Slice User Guide, UG479 v1.10[EB/OL]. 2018-03-27[2026-10-31]. https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1.

[8] XILINX. Vivado Design Suite User Guide: Implementation, UG904 v2020.2[EB/OL]. 2021-02-26[2026-10-31]. https://docs.amd.com/r/2020.2-English/ug904-vivado-implementation/About-the-Vivado-Implementation-Process.

[9] DUARTE J, HAN S, HARRIS P, et al. Fast inference of deep neural networks in FPGAs for particle physics[J]. Journal of Instrumentation, 2018, 13(07): P07027. https://doi.org/10.1088/1748-0221/13/07/P07027.

[10] YosysHQ. PicoRV32 A Size-Optimized RISC-V CPU[CP/OL]. [2026-11-01]. https://github.com/YosysHQ/picorv32. 处理器采用工程冻结实现，源码身份见各镜像清单。

[11] hls4ml Collaboration. Concepts and configuration, hls4ml documentation[EB/OL]. [2026-11-01]. https://fastmachinelearning.org/hls4ml/api/concepts.html. 采用版本 1.3.0 的冻结配置随工程提供。